Documentation de conception

Équipe GL36

Ensimag - Projet Génie Logiciel

22 janvier 2026

Résumé

Ce manuel décrit la conception et l'architecture du compilateur `decac`, en mettant l'accent sur les choix de conception (classes et dépendances) ainsi que la description des algorithmes et structures de données utilisés. Il vise à faciliter la maintenance et permettre une amélioration future du compilateur.

Table des matières

1 Architecture logicielle **3**

1.1 Vue d'ensemble 3

1.2 Organisation générale des paquetages 3

1.3 Échange d'information entre les étapes : l'arbre syntaxique abstrait (AST) . . . 4

1.4 Lien entre l'AST et l'analyse contextuelle 5

1.5 Architecture de la génération de code 8

1.5.1 Gestionnaires de ressources (Managers) 8

1.5.2 Organisation du code généré 8

1.5.3 Calcul de la taille de la pile et instructions d'initialisation 9

2 Organisation de l'analyse contextuelle **9**

2.1 Conventions de nommage et traçabilité 9

2.2 Synthèse des erreurs et vérifications 10

2.2.1 Passe 1 : Types et Hiérarchie 10

2.2.2 Passe 2 : Déclarations des Membres (Champs et Méthodes) 10

2.2.3 Passe 3 : Vérification des Corps et des Expressions 11

3 Gestion et classification des erreurs **11**

3.1 Erreurs dans un fichier `.deca` 12

3.2 Erreurs internes 12

3.3 Signalement des erreurs 12

1 Architecture logicielle

Cette section décrit l'organisation interne du compilateur `decac` et les principales interactions entre les classes qui l'implémentent.

1.1 Vue d'ensemble

Le compilateur est organisé en trois étapes principales correspondant aux phases de compilation :

- Étape A : Analyse lexicale et syntaxique → scinde le code source en token et construit l'abstrait (AST) et fait la correspondance entre les règles de la grammaire définies dans les fichiers `Lexer.g4` et `Parser.g4` et le paquetage `tree` à l'aide de `ANTLR`
- Étape B : Vérifications contextuelles → l'AST est parcouru afin de vérifier la validité sémantique du programme de décorer l'AST en conséquence
- Étape C : Génération de code → génère les instructions assembleur pour la machine `IMA`.

Ces différentes phases sont implémentées dans des paquetages distincts et sont gérées par la classe `DecacCompiler`, appelée depuis le point d'entrée du compilateur. La figure 1 schématise les interactions entre les principaux composants.

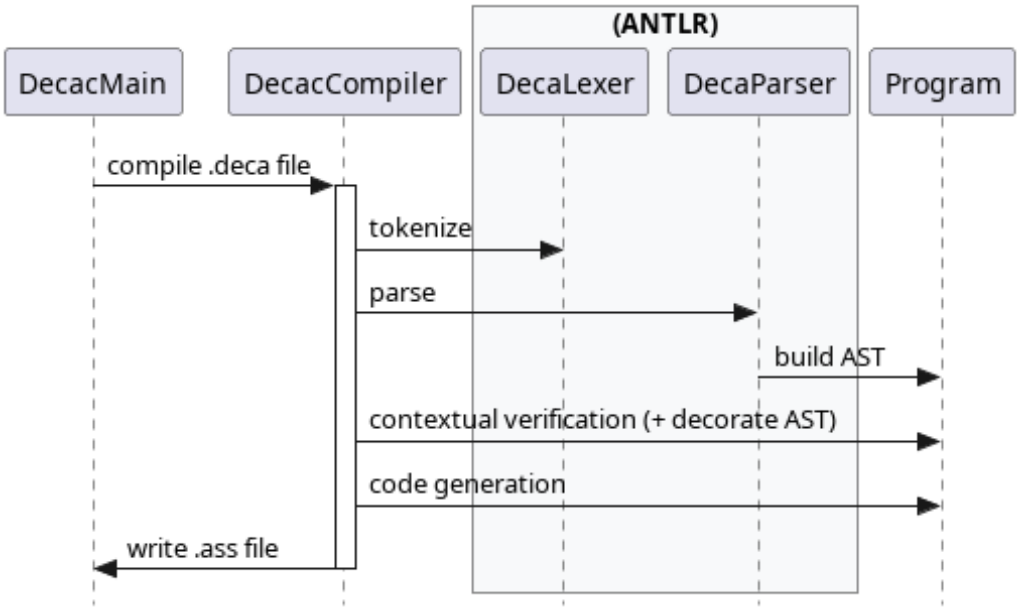


FIGURE 1 – Vue d'ensemble de la chaîne de compilation

La classe `DecacCompiler` joue un rôle central dans l'architecture du compilateur et coordonne l'exécution des différentes phases de compilation.

1.2 Organisation générale des paquetages

Le code source est organisé autour de paquetages distincts :

- `syntax` : classes (`AbstractDecaParser`, `AbstractDecaLexer`)qui permettent de faire le lien entre le compilateur et les classes générées par `ANTLR` selon les règles écrites dans les fichiers .g4 + gestionnaires d'erreurs

- `tree` : classes permettant la représentation des nœuds de l'arbre abstrait
- `context` : toutes les classes (Types, Environnements, Définitions) utilisées lors de la décoration et vérification contextuelle + gestionnaire d'erreurs contextuelles (`ContextualError`)
- `codegen` : gestionnaires de pile, registres, labels, table des méthodes, et erreurs à l'exécution
- `ima` : représentation des instructions assembleur IMA
- `tools` : classes utilitatires (table des symboles, affichage de l'AST, erreurs internes).

1.3 Échange d'information entre les étapes : l'arbre syntaxique abstrait (AST)

Les données générées par le compilateur sont transmises au travers de l'arbre syntaxique abstrait. Celui-ci est construit lors de l'analyse syntaxique, enrichi lors de l'analyse contextuelle (ajout d'informations de type et de définitions), puis parcouru lors de la génération de code. Chaque phase effectue un ou plusieurs parcours de l'arbre y ajoute des nœuds (`ConvFloat`) ou des informations supplémentaires (types, définitions) mais n'en modifie pas la structure générale.

L'implémentation de l'AST suit le patron de conception interprète, qui permet de représenter chaque règle de la grammaire abstraite par une hiérarchie de classes Java. Chaque nœud de l'arbre est codé par une instance d'une classe dérivée de `Tree`, les fils éventuels étant stockés dans des champs de l'objet.

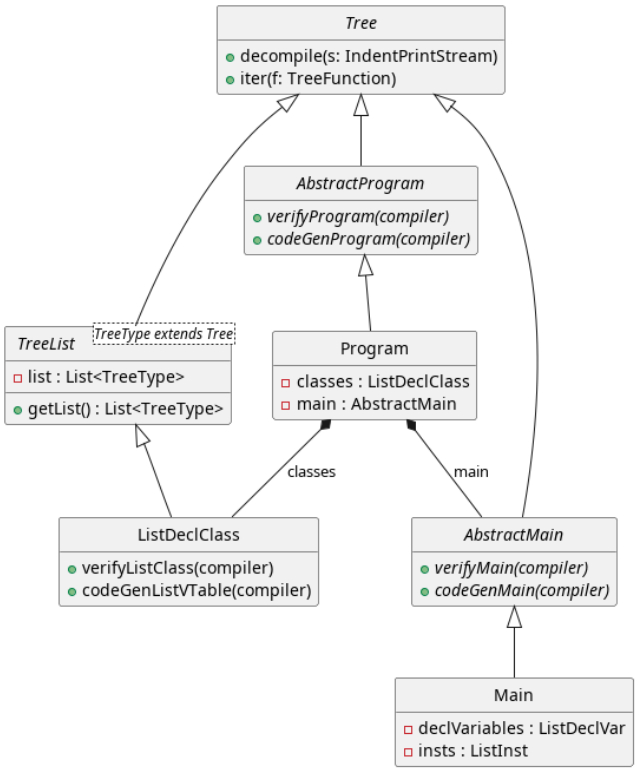


FIGURE 2 – Structure de haut niveau de l'AST

Les points essentiels de cette correspondance sont les suivants :

- Chaque terminal de la grammaire est représenté par une classe concrète, tandis que chaque non-terminal correspond à une classe abstraite dont héritent les classes concrètes appropriées.

- Les ensembles de nœuds (listes de variables, méthodes, champs, paramètres) sont représentés par des classes héritant de `TreeList`, paramétrées sur le type des éléments.
- L'héritage des classes reflète les relations dans la grammaire abstraite
- Chaque parcours de l'arbre (décompilation, vérification contextuelle, génération de code) correspond à une méthode abstraite déclarée dans la super-classe abstraite appropriée.

Cette approche vise à minimiser les erreurs liées à un traitement au cas par cas, comme le ferait un code utilisant `instanceof` ou des tests sur les opérateurs, et facilite l'extension du compilateur avec de nouveaux types de nœuds.

1.4 Lien entre l'AST et l'analyse contextuelle

Un objectif majeur auquel doit répondre l'architecture du compilateur est de distinguer la représentation syntaxique du programme de l'interprétation sémantique de ses éléments, tout en permettant l'interaction des deux. Les classes `DeclClass`, `DeclMethod`, `DeclField` et `DeclParam` sont instanciées par le parser lorsque des déclaration de la bonne forme sont reconnues dans le code source. Ce sont les nœuds de l'AST.

Le paquetage `context` dépend d'une hiérarchie de définitions centrée autour de la classe abstraite `Definition`. La spécialisation de `Definition` en `TypeDefinition` et `ExpDefinition` permet de distinguer clairement les entités relevant du système de types (comme les classes) de celles qui sont manipulables dans des expressions.

Étant donné que la vérification contextuelle a aussi pour but de préparer l'AST pour la génération de code, la classe `ClassDefinition` encapsule non seulement les informations propres à une classe (nombre de champs, nombre de méthodes, super-classe), mais également son environnement de membres dans la classe `EnvironmentExp` (lié aux environnements des super-classes dans une liste chaînée). De même, les classes `MethodDefinition`, `FieldDefinition` et `ParamDefinition` peuvent stocker des index servant lors de la génération de code. Toutes ces relations sont illustrées dans la figure 3.

Remarque : L'implémentation de `EnvironmentExp` repose sur une `HashMap` qui associe les symboles à leurs définitions. Ce choix garantit une complexité de recherche en O(1) en moyenne ce qui est souhaitable puisque la recherche de noms est l'opération la plus fréquente lors de la décoration de l'AST et il n'y a pas d'ordre particulier à respecter. L'unicité est assurée manuellement lors de l'insertion dans l'environnement (par `SymbolTable`)

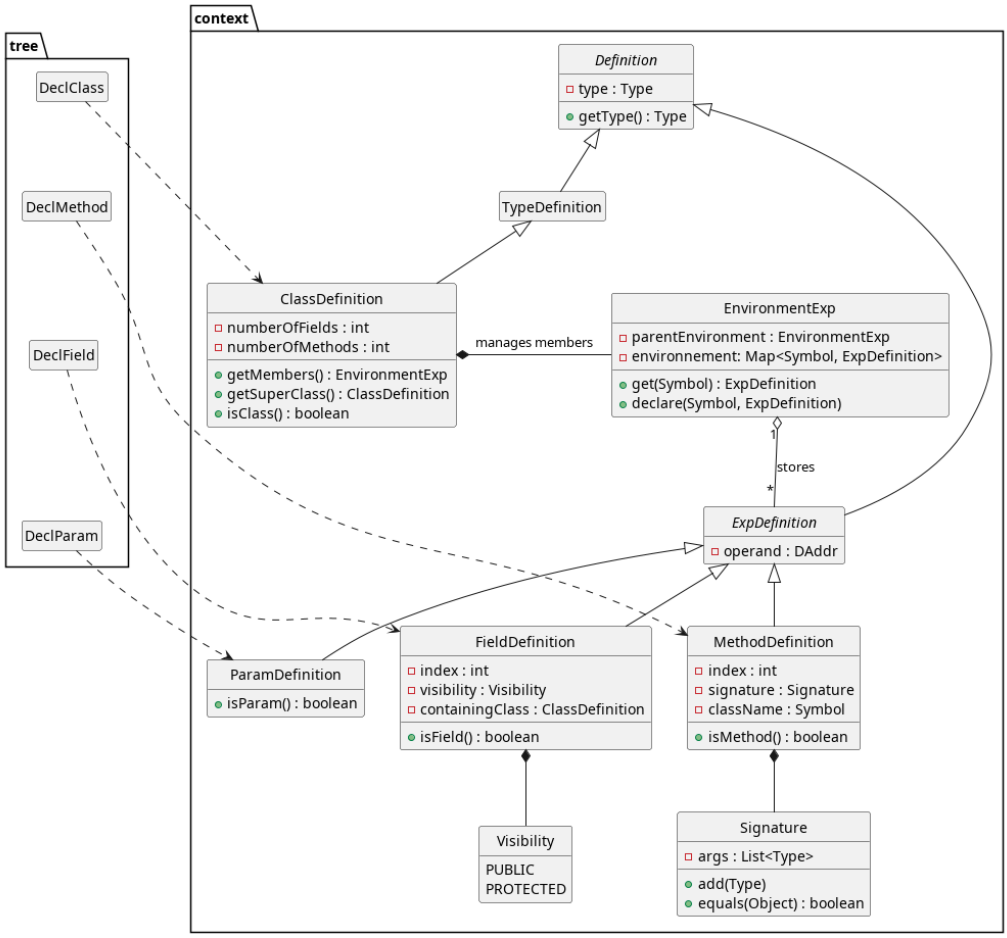


FIGURE 3 – Relations entre les nœuds de l'AST et les définitions contextuelles

Par exemple, lorsqu'une déclaration de méthode est rencontrée, une instance de `MethodDefinition` est créée et associée au nœud `DeclMethod` correspondant lors d'une passe de l'analyse contextuelle (`verifyMethodMembers`). Cette association permet d'ajouter à la déclaration le type de retour, ses paramètres et sa visibilité, des objets qui n'ont un sens qu'au moment de la vérification contextuelle.

../../../src/main/java/fr/ensimag/deca/tree/DeclMethod.java

```
102         MethodDefinition methodDef = new MethodDefinition(type, getLocation
    (), sig, index);
103         methodDef.setClassName(currentClass.getType().getName());
104         try {
105             currentClass.getMembers().declare(methodName.getName(),
    methodDef);
106         } catch (DoubleDefException e) {
107             throw new ContextualError("Method ’" + methodName.getName() + "’
    is already defined in class ’" + currentClass.getType().getName().
    getName() + "’.", getLocation());
108         }
109
110         methodName.setDefinition(methodDef);
```

Code 1 – Extrait de la classe `DeclMethod` montrant l'association avec `MethodDefinition`

L'analyse sémantique ne se limite pas aux structures de classes ; elle s'applique à chaque expression pour garantir la cohérence entre les types évalués et les types attendus. Comme l'illustre la figure 4, ce processus repose sur deux mécanismes :

- La décoration des nœuds : Chaque nœud héritant de `AbstractExpr` dispose d'un champ `type`. Lors de la vérification, ce champ est lié à l'une des instances uniques gérées par l'objet `EnvironmentType` (comme `INT` ou `FLOAT`), permettant des comparaisons de types efficaces par référence.
- La résolution des identificateurs : Contrairement aux littéraux qui portent leur type intrinsèquement, un `Identifier` doit interroger les environnements contextuels (`EnvironmentExp`) pour obtenir une `Definition`. Ce lien permet d'accéder aux métadonnées comme l'index en mémoire ou la visibilité d'un champ ou d'une méthode.

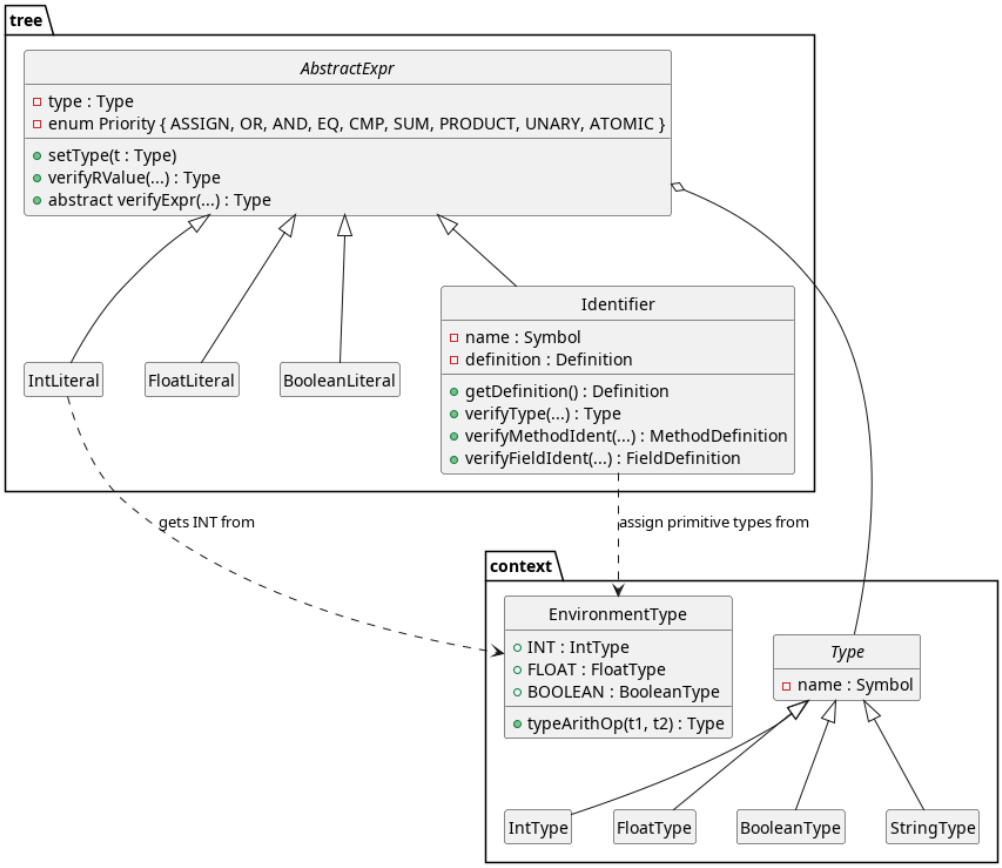


FIGURE 4 – Relations entre les nœuds d'expression de l'AST et les définitions contextuelles

Enfin, l'analyse contextuelle peut insérer un nœud `ConvFloat` : si une expression attend un flottant mais reçoit un entier, le compilateur encapsule le nœud original pour rendre la promotion explicite. Cette architecture pourrait permettre, par exemple, d'ajouter des transtypages implicites (comme une troncation d'entier) sans modifier la structure du compilateur existante, bien que ce type de comportement ne soit pas prévu dans le langage Deca.

Remarque : Le diagramme de la figure 4 a été simplifié pour mettre en évidence les relations de décoration. L'attribut `Priority` de `AbstractExpr` est quant à lui utilisé lors de la phase de décompilation pour assurer une gestion correcte de la priorité des opérateurs lorsqu'il faut rajouter des parenthèses.

1.5 Architecture de la génération de code

L'étape de génération de code transforme l'AST décoré en instructions pour la machine virtuelle `IMA`. Cette phase s'appuie sur une série de gestionnaires qui centralisent le code et en facilitent la maintenance.

1.5.1 Gestionnaires de ressources (Managers)

Chaque gestionnaire est responsable d'un (ou plusieurs) aspect spécifique du processus de génération de code :

- → `VTableManager` et `VTableLayout` : Ces classes construisent la structure des tables des méthodes virtuelles. Elles prennent en charge l'héritage en copiant la table de la super-classe avant d'ajouter ou de mettre à jour les entrées pour les méthodes redéfinies. Le gestionnaire réserve également les slots nécessaires en base de pile globale (`GB`). Ce gestionnaire est aussi chargé de générer les instructions relatives à la classe `Object`
- → `RuntimeErrorManager` : Afin d'optimiser la taille du code, ce gestionnaire centralise les types d'erreurs (*division_by_zero*, *null_dereference*, *heap_overflow*, etc.). Il ne génère le bloc d'instructions `WSTR` et `ERROR` associé, qu'une seule fois en fin de fichier, et uniquement si une instruction susceptible de provoquer l'erreur a été rencontrée durant le parcours de l'AST.
- → `LabelManager` : Il garantit l'unicité des étiquettes (`Label`) pour les structures de contrôle (`if`, `while`), les instructions `cast` et `instanceof`, ainsi que pour les méthodes et l'initialisation des champs. Chaque étiquette est suffixée d'un entier incrémenté par un compteur dédié, évitant ainsi toute collision de noms.
- → `RegisterManager` : Ce gestionnaire contrôle l'utilisation des registres de la machine (`R2` à `R15`). Il fournit le prochain registre disponible en suivant un principe d'allocation séquentielle. Cette stratégie, s'appuyant sur un simple compteur (et non pas une structure de données complexe), est rendue possible par la structure récursive de l'AST qui garantit que les registres seront libérés dans l'ordre inverse de leur allocation. Le gestionnaire assure aussi le débordement (*spilling*) : si tous les registres sont occupés, il génère automatiquement une instruction `PUSH` pour sauvegarder le contenu sur la pile, puis un `POP` pour le restaurer. Enfin, il gère la sauvegarde du contexte lors des appels de méthodes avec les méthodes `saveRegisters` et `restoreRegisters`.
- → `StackManager` : Il suit l'occupation de la pile globale (`GB`) pour les variables globales et les tables des méthodes, ainsi que de la pile locale (`LB`) pour les variables locales et les paramètres. Il permet de calculer avec précision la taille maximale nécessaire à l'exécution du programme.

1.5.2 Organisation du code généré

L'un des défis de la génération de code réside dans le fait que certaines informations, comme la taille totale de la pile, ne sont connues qu'une fois l'intégralité de l'AST parcourue. Pour résoudre ce problème sans effectuer de passes supplémentaires, le compilateur utilise plusieurs instances de `IMAProgram`. Le code est généré dans des blocs distincts (`head` pour les tables des méthodes, `body` pour le programme principal, et `preamble` pour les instructions de contrôle). Ces blocs sont concaténés dans l'ordre approprié par la classe `Program`. Un résumé de cette organisation est présenté dans la figure 5.

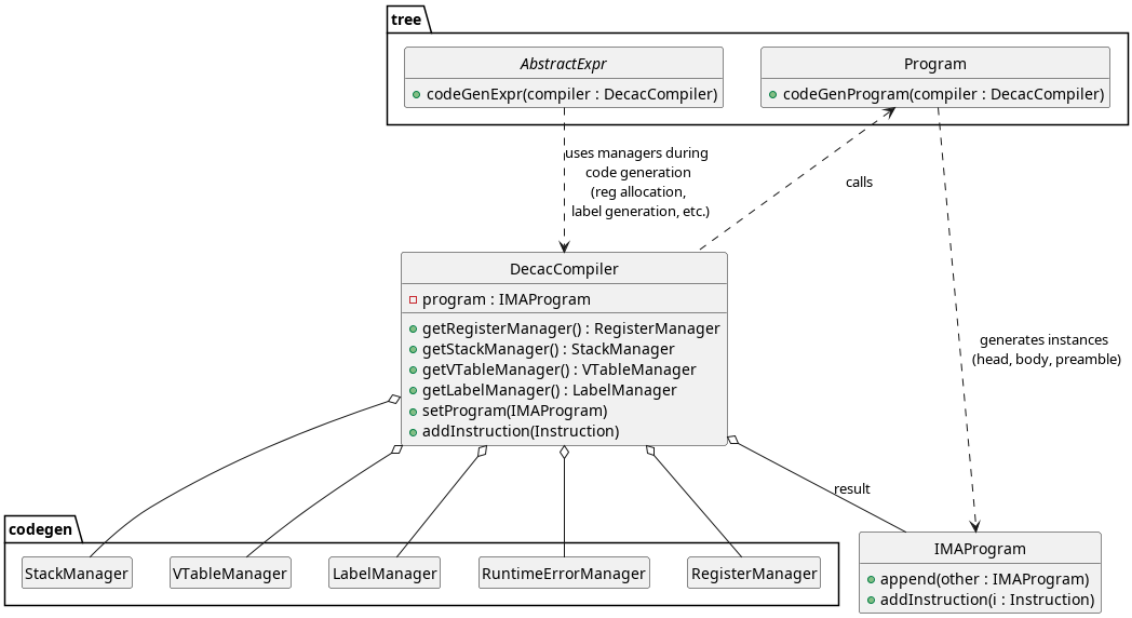


FIGURE 5 – Architecture des gestionnaires de la génération de code

1.5.3 Calcul de la taille de la pile et instructions d'initialisation

Le début du fichier assembleur doit impérativement contenir les instructions `TSTO` (*Test Stack Overflow*) et `ADDSP` (*Add Stack Pointer*) pour éviter des pertes de données ou des crashs imprévus. Grâce au compteur du `StackManager`, le compilateur récupère les dimensions finales de la pile. La valeur passée à `TSTO` doit anticiper le pire scénario d'occupation de la mémoire pour garantir un arrêt propre en cas de dépassement. Le calcul combine les données de plusieurs gestionnaires :

$$\texttt{TSTO} = \texttt{StackSize} + \texttt{NbSavedRegs} + \texttt{MaxTempUsage}$$

- `StackSize` : Emplacements pour les variables globales et les VTables (géré par le `StackManager`).
- `NbSavedRegs` : Nombre maximal de registres sauvegardés lors des appels de méthodes (fourni par le `RegisterManager`).
- `MaxTempUsage` : Pic d'utilisation de la pile lors de l'évaluation d'expressions complexes ou du débordement de registres (calculé par le `RegisterManager`).

Cette valeur est vérifiée par une instruction `BOV` (*Branch on Overflow*) qui pointe vers une étiquette d'erreur d'exécution générée par le `RuntimeErrorManager`.

2 Organisation de l'analyse contextuelle

L'analyse contextuelle est divisée en trois passes afin de résoudre les problèmes de forward reference. Ces différentes passent permettent de recenser les types avant leur utilisation comme types de champs ou de paramètres et de déclarer l'ensemble des membres (champs et méthodes) avant de vérifier la cohérence sémantique du corps des méthodes/classes.

2.1 Conventions de nommage et traçabilité

Afin de maintenir une correspondance directe avec le polycopié de référence, nous avons ajouté dans la Javadoc de chaque méthode le numéro de la passe et la règle associée lorsque cela était

pertinent. L'implémentation suit la convention de nommage suivante :

`verify[Nom_de_classe][Passe]`

Les suffixes de passes ont été choisis pour refléter fidèlement la sémantique de l'étape :

- la Passe 1 identifie les classes et la hiérarchie
- la Passe 2 (`Members`) vérifie les signatures des membres (champs et méthodes)
- la Passe 3 (`Body`) traite le code exécutable et les initialisations. Par souci de clarté, ces noms ont été adaptés lorsque le contexte l'exigeait (par exemple, on utilise `verifyFieldRValue` en passe 3 d'une déclaration de champ (`DeclField`), plutôt que `verifyBody`, qui n'avait pas beaucoup de sens).

2.2 Synthèse des erreurs et vérifications

Conformément aux spécifications de [SyntaxeContextuelle] du polycopié, chaque règle de la grammaire attribuée impose des conditions dont le non-respect déclenche la levée d'une `ContextualError`. Le tableau ci-dessous recense les raisons de ces erreurs ainsi que les messages d'erreur associés.

2.2.1 Passe 1 : Types et Hiérarchie

Règle	Opération partielle / Condition	Message d'erreur
(1.3)	Nom de classe déjà présent dans `env_types`.	Type <s> is already defined.
(1.3)	Super-classe non trouvée dans l'environnement.	Superclass <name> of class <C> is not defined.
(1.3)	La super-classe n'est pas de type `class`.	Superclass <name> is not a class type.
(0.2)	Identificateur de type inconnu.	Type <name> not found

2.2.2 Passe 2 : Déclarations des Membres (Champs et Méthodes)

Règle	Opération partielle / Condition	Message d'erreur
(2.5)	Déclaration d'un champ de type `void`.	Field type cannot be void
(2.5)	Nom de champ identique à une méthode héritée.	Field name collides with an inherited method
(2.5)	Double déclaration d'un champ dans la même classe.	Field <name> already defined in class
(2.7)	Méthode déjà définie dans la classe courante.	Method '<name>' is already defined in class '<C>'.
(2.7)	Incompatibilité de signature ou type de retour (*override*).	Method '<name>' has incompatible signature/type.
(2.9)	Paramètre de méthode déclaré comme `void`.	Parameter cannot be of type void
(2.9)	Double déclaration d'un paramètre.	Parameter <name> is already defined

2.2.3 Passe 3 : Vérification des Corps et des Expressions

Règle	Opération partielle / Condition	Message d'erreur
(3.17)	Variable locale déclarée avec le type `void`.	Variable type cannot be void
(3.17)	Double déclaration d'une variable locale.	Variable <name> is already defined
(3.22)	Condition de contrôle non booléenne.	Condition must be a boolean. Found : <type>
(3.24)	Expression de `Return` de type `void`.	Return expression cannot be of void type
(3.28)	Incompatibilité d'affectation (`assign_compatible`).	Incompatible types in Rvalue... cannot assign <T1> to <T2>
(3.31)	Paramètre de `print` invalide.	Invalid parameter type (expected int, float, or string)
(3.33)	Types incompatibles pour une opération arithmétique.	Incompatible types for arithmetic operation
(3.37)	`UnaryMinus` sur un type non numérique.	Operand of UnaryMinus must be of numeric type
(3.39)	Échec de la compatibilité du `Cast`.	Cannot cast from <T1> to <T2>
(3.42)	`New` utilisé sur un type non-classe.	New can only be used with class types
(3.43)	`This` utilisé dans le `main` (hors classe).	'this' cannot be used outside of a class
(3.51)	Utilisation invalide du Modulo.	Incompatible types for modulo operation
(3.63)	Opérateur `Not` sur un type non-booléen.	Operand of Not ('!') must be of boolean type
(3.66)	Violation des droits d'accès au champ protégé.	Visibility error for protected field
(3.67)	Identificateur inconnu dans l'environnement.	Identifier <name> not found
(3.71)	Mauvais nombre d'arguments lors de l'appel.	Expected <n> args, got <m>

3 Gestion et classification des erreurs

Le compilateur distingue plusieurs catégories d'erreurs, conformément aux spécifications fournies. Chaque étape de compilation possède son propre mécanisme de détection mais aussi ses propres erreurs (et exceptions) pour permettre de facilement identifier la source du problème lorsqu'une erreur survient.

3.1 Erreurs dans un fichier `.deca`

Il s'agit des erreurs dues à un programme `.deca` incorrect, qui doit alors être rejeté par le compilateur. Ces erreurs peuvent être détectées à différentes étapes de la compilation (étapes A et B principalement). Dans ce cas, la compilation est immédiatement interrompue dès la première erreur détectée, sans tentative de récupération d'erreur.

3.2 Erreurs internes

Les erreurs internes sont dûes à des défaillances du compilateur lui-même (bugs, mauvaises implémentations). Elles entraînent l'arrêt immédiat du compilateur, mais ne devraient pas se produire lors d'une utilisation normale.

3.3 Signalement des erreurs

La détection et la propagation des erreurs reposent sur le mécanisme d'exceptions en Java, conformément aux instructions. Le compilateur s'arrête à la première erreur rencontrée.

- `RecognitionException` pour les erreurs de l'analyse syntaxique
- `ContextualError` pour les erreurs détectées lors de l'analyse contextuelle
- `CLIException` pour les erreurs liées à la ligne de commande
- `DecacFatalError` pour les erreurs d'entrée/sortie
- `DecacInternalError` pour les erreurs internes.

Les messages d'erreur respectent le format imposé :

```
<nom-fichier>:<no-ligne>:<no-colonne>: <description>
```

Remarque : Les avertissements n'ont pas été implémentés dans la version actuelle du compilateur. Un avertissement est affiché lorsqu'un programme `.deca` est correct mais pour lequel le compilateur détecte une situation éventuellement problématique. Dans ce cas le compilateur pourrait générer le code assembleur, mais signalerait le problème potentiel à l'utilisateur.trice par un message d'avertissement.